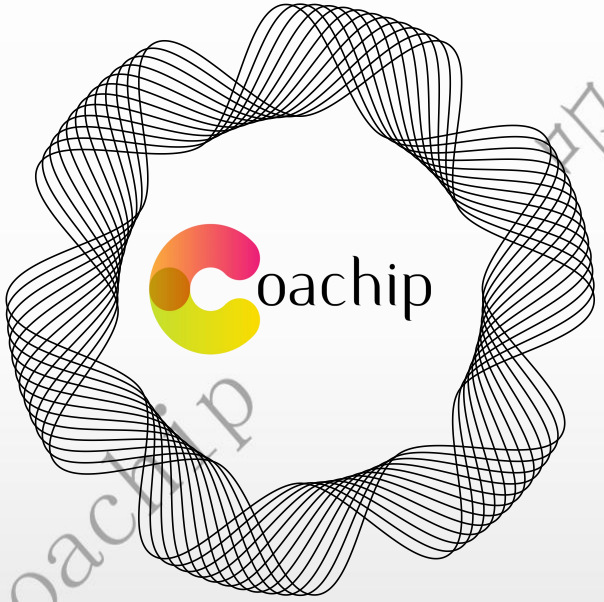


SV接口的驱动和采样

叩特电子Coachip

课程内容及目标

课程内容：

- 接口同步
- 接口信号采样
- 接口信号驱动
- 接口中的双向信号
- program中不能使用always的原因
- 时钟发生器

目标：

- 学习并掌握接口的使用



叩特电子Coachip

接口同步

- 用verilog的@和wait来同步测试平台中的信号.

```
program automatic test(bus_if.TB bus);
    initial begin
        @bus.cb;                          //clock模块中时钟的有效沿触发
        repeat(4) @bus.cb;                //等待4个有效时钟沿
        @bus.cb.grant;                    //监测grant信号的变化(上升，下降)
        @(posedge bus.cb.grant);  //监测grant信号的上升沿
        @(negedge bus.cb.grant);  //监测grant信号的下降沿
        wait (bus.cb.grant==1);       // 监测一个电平值, 若监测不到, 则一直等待
        @(posedge bus.cb.grant or negedge bus.reset); //思考:这个是等待什么?
    end
endprogram
```

叩特电子Coachip

接口信号采样

- 当在DUT arb中驱动arbif.grant：
  test对clocking cb中的输入信号grant进行采样。
  采样输出为arbif.cb.grant

```
clocking cb @(posedge clk);
    output request;
    input grant;
endclocking

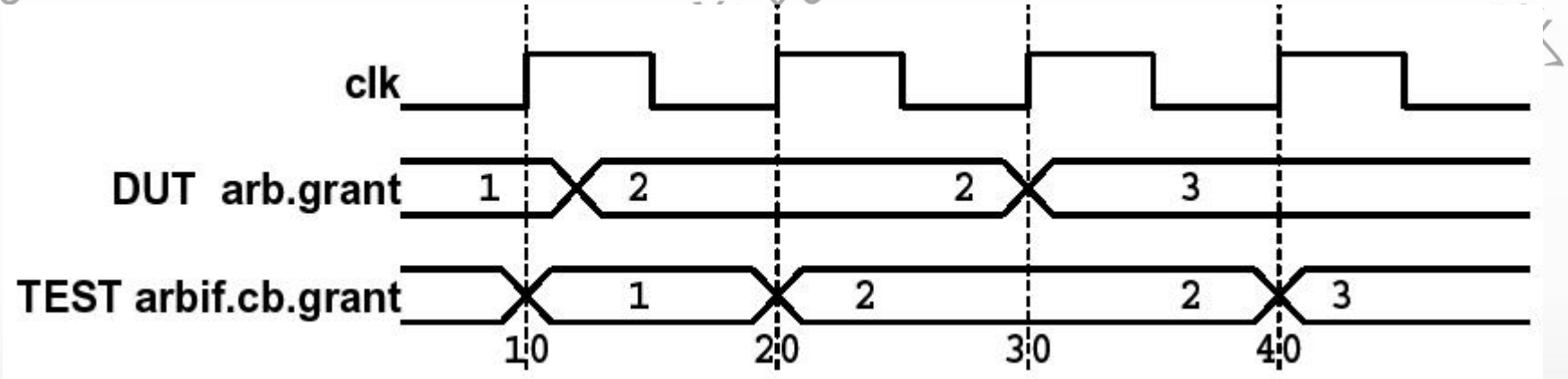
program test(arb_if.TEST arbif);
    initial begin
      $monitor("@%0d: grant=%h", $time, arbif.cb.grant);
      #50;
    end
endprogram
```

```
module arb(arb_if.DUT arbif);
      initial begin
          arbif.grant = 1; // @ 0ns
        #12 arbif.grant = 2; // @ 12ns
        #18 arbif.grant = 3; // @ 30ns
      end
endmodule
```

叩特电子Coachip

接口信号采样

DUT的信号输出grant，clock block的cb.grant输出如波形所示。



在30ns处，DUT的grant的值在时钟沿跳变，同时TEST的cb.grant也在该时钟沿触发输出，但此时它采样的值还是该时刻之前的值2。

即特电子Coachip

接口信号驱动

- 在test中驱动同步信号arbif.cb.request，在DUT中监测arbif.request的输出

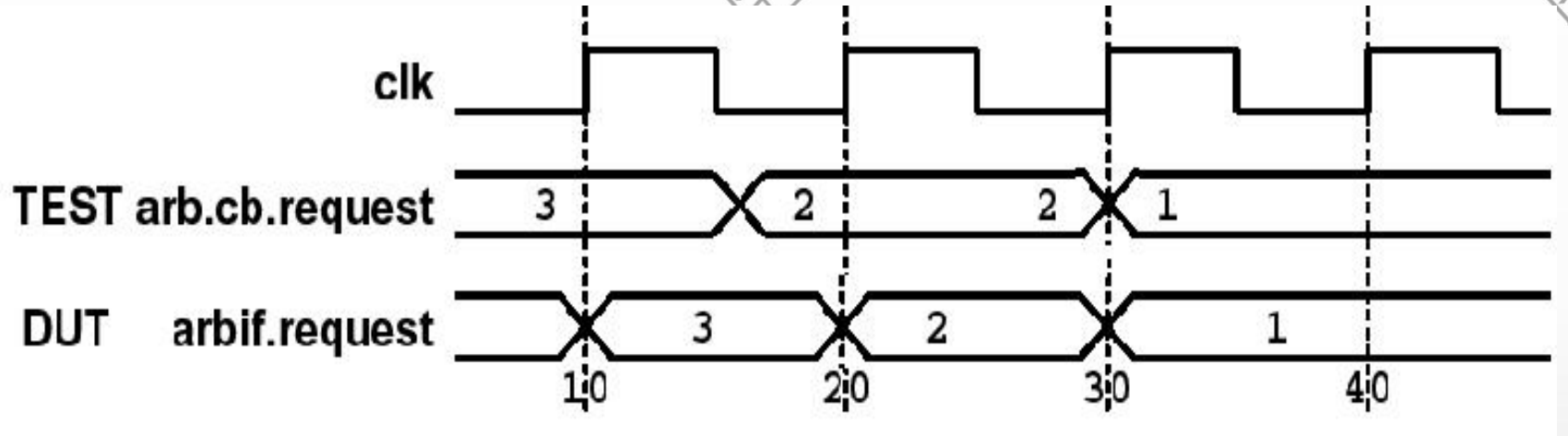
```
program test(arb_if.TEST arbif);
  initial begin
    # 7 arbif.cb.request <= 3; // @ 7ns
    #10 arbif.cb.request <= 2; // @ 17ns
    #13 arbif.cb.request <= 1; // @ 30ns
    #15 $finish;
  end
endprogram

module arb(arb_if.DUT arbif);
   initial
      $monitor("@%0t: req=%h", $time, arbif.request);
endmodule
```

叩特电子Coachip

接口信号驱动

test中arb.cb.request，DUT中request的波形如图



注意：30ns处arb.cb.request的变化立刻在上升沿被传送到arb.request上。因为在clocking block中输出信号的默认延迟为#0，软件在编译时， clocking block中的输出信号要晚于被采样信号的变化，但在波形上观察不出来。

接口信号驱动

- 总结：

在clocking block中，采样（input）和驱动（output）信号遵循如下原则：

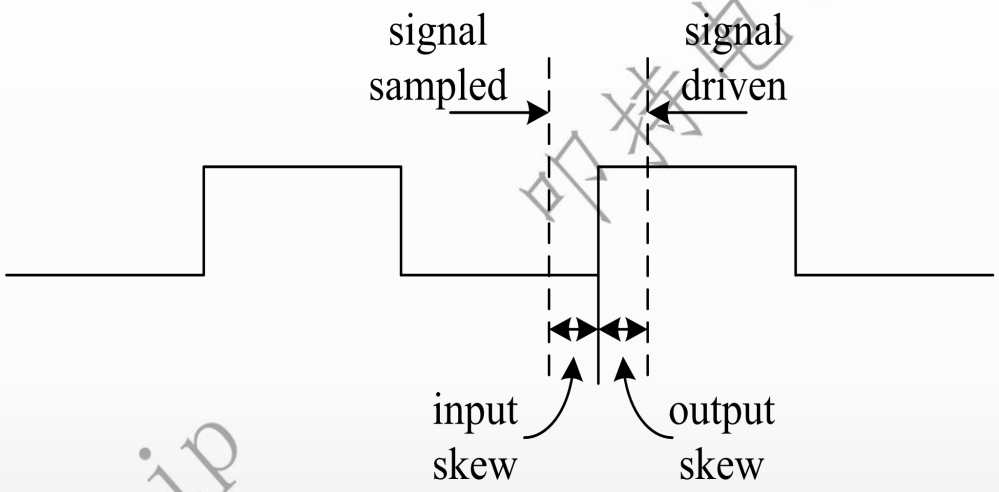
同步后的**采样信号**，都是前一个状态的值。

同步后的**驱动信号**，都是当前状态的值。

叩特电子Coachip

clocking block中的setup和hold

- 输入偏差一般指输入信号被采样时刻先于采样信号发生的时间间隔，一般这个偏差默认为1step；
- 输出偏差一般指采样事件发生时刻到输出被驱动到输出端口的时间间隔，这个偏差默认为0；
- clocking blocking的输入和输出可以通过设置延迟来模拟实际的输入和输出偏差，这种偏差都是相对于当前采样事件而定的；



即特电子Coachip

接口中的双向信号

```
interface master_if(input bit clk);
    wire[7:0] data;
    clocking cb@(posedge clk);
        inout data;
    endclocking
    modport TEST(clocking cb);
endinterface

program test(master_if mif);
    initial begin
        mif.cb.data <='z;        //高阻态
        @mif.cb;
        $display(mif.cb.data);  //从总线读取
        @mif.cb;
        mif.cb.data <=7'h5a;    //驱动总线
        @mif.cb;
        mif.cb.data <='z;        //释放总线
    end
endprogram
```

由于SV的用户参考手册中没有明确定义如何驱动接口中的异步双向信号，故可用以下两种方法：

1、连续赋值语句

2、虚接口 `virtual interface`

叩特电子Coachip

Program中不能使用always的原因

always块在program中不支持(编译会报错)

testbench中，程序的运行是有序性的：初始化，驱动激励信号，收取设计对激励的反馈，最后结束。当最后一个initial模块结束后，这个仿真也结束了,就像执行了$finish一样。而 always 语句是没有结束的，需要$exit来指定结束；

如果确实需要用一个always块, 可以使用“initial forever”来完成相同的事情;所以initial forever在testbench中可以使用，功能类似于always.

即特电子Coachip

时钟发生器

```
program bad_generator (output bit clk, out_sig);
      bit clk=0, out_sig=0;
      initial
        forever #5 clk <= ~clk;
      initial
        forever @(posedge clk)
          out_sig <= ~out_sig;
endprogram
```

上例将时钟的产生定义到了program中，这样就会产生速度问题，时钟的翻转和信号变化同时的，应该判定谁的变化在前？可能会引起竞争状态。 所以应该在module或者class中声明clk

```
module clock_generator (output bit clk);
      bit clk = 1;
      always #5 clk = ~clk; // Use blocking assignment
endmodule
```

叩特电子Coachip

本章回顾

- 接口同步
- 接口信号采样
- 接口信号驱动
- 接口中的双向信号
- program中不能使用always的原因
- 时钟发生器

叩特电子Coachip

Q&A

昂特电子Coachip